对外发布与多用户访问方案 V1.0

文档定位：本文是《电商销量预测系统 · 部署运维手册 V1.0》的**补充**，只回答一个问题 ——

如何把「本机 127.0.0.1 上跑得通的应用」变成「多个人能同时访问和使用的服务」。

手册已覆盖的安装、启动、备份、升级、回滚（§2 / §3 / §6 / §7 / §12）**不重复**，只做交叉引用。

编写基线：工作区当前代码 + 手册 V1.0 + 本机实测数据。文中标注 [实测] 的数字取自手册 §10.1 的 2026-10-12 基线；标注 [待实测] 的表示**换机器后必须重测**，不能照搬。

目 录

0. 结论先行 3

0.1 三条可行路径 3

0.2 一句话风险提示 3

1. 现状：本机形态 → 多人可访问，差在哪 3

2. 访问方式（三选一） 4

方案 A：局域网内网访问 4

方案 B：公网 + 域名 + HTTPS（推荐正式发布用） 5

方案 C：临时隧道（仅演示，不建议承载真实数据） 6

三种方式对比 7

3. 运行环境要求 7

3.1 硬件（引手册 §2.1） 7

3.2 操作系统 8

3.3 依赖 8

3.4 网络与端口 8
3.5 磁盘与数据 9
4. 部署步骤（生产形态） 9
4.1 Linux 服务器（照手册执行） 9
4.2 Windows 服务器 10
4.3 关键：前端必须构建，不能跑 dev server 11
5. 权限控制 11
5.1 第一层：认证（你是谁） 11
5.2 第二层：授权与数据隔离（你能碰谁的数据） 12
5.3 第三层：功能门控 12
5.4 🔴 公开暴露前必改清单 12
6. 🔴 公开前必须处理的阻断项 13
6.1 【阻断级】游客之间没有任何数据隔离 13
6.2 【高】匿名用户可以无限消耗算力 15
6.3 【高】默认管理员账号 16
6.4 【中】配置项 16
6.5 【中】单进程带来的并发上限 16
7. 容量与并发（单 worker 的现实） 17
7.1 为什么必须是单进程 17
7.2 实测并发能力（手册 §10.1） 17
7.3 扩容路径（按优先级） 18
8. 运维要点（交叉引用手册） 18
9. 上线检查清单 19
附录：与上游文档的关系 20

0. 结论先行

0.1 三条可行路径

路径	访问方式	适合场景	是否适合承载真实业务数据
A 局域网（内网）	`http://<内网 IP>/`	同一办公室 / 机房内几个人用	✅ 可以
B 公网 + 域名 + HTTPS	`https://your-domain.com/`	正式对外发布（推荐）	⚠ **需先修 §6.1**
C 临时隧道	隧道服务给的随机域名	只给他人**临时看演示**	✕ 不建议

0.2 一句话风险提示

🟤 **当前代码在「多人可访问」形态下有一个必须先修的问题：游客上传的数据是「匿名共享」的** ——

任何访客（**甚至已登录的普通用户**）都能看到其他访客上传的数据与任务（§6.1 有代码级证据）。

单人自用时看不出来；一旦公开，**别人上传的真实销售数据会被其他人看到。**

处理方式二选一：① **上线前关闭游客免登录链路**（最省事、且能让「匿名共享」回归示例数据的本意）

或 ② **给游客发匿名会话标识做隔离**（保留免登录体验）。详见 §6.1。

---

1. 现状：本机形态 → 多人可访问，差在哪

维度	现在的状态	多人访问需要变成
后端监听	`--host 127.0.0.1`（手册 §3 / §6.2 的安全默认）	前面挂**反向代理**（推荐），或改 `0.0.0.0`（不推荐）

维度	现在的状态	多人访问需要变成
前端	Vite **dev server**（`:5173`，仅开发用）	`vite build` 出**静态产物**，由 Nginx 托管 —— **dev server 绝不能对外**
跨域	`CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173`	追加真实访问来源（同源部署时理论上可不依赖 CORS，但显式配置更稳）
身份与隔离	注册用户之间**已隔离**（`user_id` 归属校验）；**游客之间未隔离**	补上游客隔离，或取消游客通道（§6.1）
容量与配额	单进程 `--workers 1`；**除登录外无任何配额**	明确并发上限 + 至少给匿名访问加闸（§6.2 / §7）

📌 **注册用户之间本来就是隔离的** —— 这一点不用改。归属判定有唯一真源 `app/services/result_service.py::_owned_task()`，越权返回 `40301`（`NFR-SEC-05`）。

问题**只出在游客哨兵 `user_id = 0` 这个共享桶**上。

---

2. 访问方式（三选一）

方案 A：局域网内网访问

适用：同一办公室 / 机房 / 校园网内，几个人同时用。

做法：

1. 后端仍可只监听本机，由 Nginx 对外监听 `80`：

```
listen 80;
server_name _;          # 或内网域名
root /srv/sales-forecast/web/dist;
```

2. 查本机内网 IP：`ipconfig`（Windows）/ `ip addr`（Linux）
3. 其他人访问 `http://<内网 IP>/`

优点：零域名、零证书、零备案；数据不出内网。

缺点：出了这个网就用不了（家庭宽带 / 4G 访问不到）。

方案 B：公网 + 域名 + HTTPS（推荐正式发布用）

架构（单机即可，无需额外组件）：

```
浏览器 ──HTTPS──> Nginx (443) ──┬── 静态文件 web/dist/       (前端)
                                └── /api/ /health /docs ... ──> 127.0.0.1:8000 (后端)
```

为什么必须是这个形状：

- 后端**只监听本机**，不直接暴露 —— 面小得多；
- 前端由 Nginx 出静态文件，**取代 Vite 的 dev server 和它的 `/api` 代理**；
- 一个域名一个端口，**同源**，省掉大半跨域问题。

前置条件：

项	要求
服务器	一台有公网 IP 的云主机（**不要求 GPU**，见 §3）
域名	已解析到该公网 IP 的 A 记录（如 `forecast.example.com`）
端口	安全组 / 防火墙放行 **80 与 443**；**不要**放行 8000

项	要求
证书	Let's Encrypt（certbot 一行命令）或云厂商免费证书
合规	中国大陆服务器 + 域名对外提供 Web 服务，需完成 **ICP 备案**；未备案可用非 80/443 端口或境外节点，但体验与合规性都更差

具体步骤：直接照手册 **§3「生产部署」** 执行即可 —— 那里已有完整的 `vite build` 命令、systemd 单元文件（§6.2）和 Nginx 站点配置（含 `client_max_body_size 25m`、`proxy_read_timeout 90s` 等关键项）。本文只强调三处**多人访问特有的差异**：

1. `server_name` 从 `_` 改成真实域名；
2. `CORS_ORIGINS` 追加 `https://forecast.example.com`；
3. `proxy_read_timeout` 保持 `> 90s` —— 多人排队时单任务可能比单人慢，别让 Nginx 先掐断。

方案 C：临时隧道（仅演示，不建议承载真实数据）

用 cpolar / ngrok / frp 之类把本机端口映射成公网随机域名。**适合「现在给同事看一眼」**。

⚠ **三条限制必须知道**：

1. 免费版域名随机且**每次重启都变**，没法当作正式地址；
2. 流量经第三方中转 —— **上传的业务数据会经过他人服务器**；
3. 仍然绕不开 §6.1 的游客隔离问题（**反而更危险**：公网可访问 + 无隔离）。

> 📌 **如果只是自己机器上演示给面试官看**：用方案 C + 明确「请勿上传真实数据」可以接受；
>
> **如果是要给真实用户用**：必须走方案 B。

三种方式对比

维度	A 内网	B 公网+域名	C 临时隧道
谁能访问	同网络内	任何人	拿到链接的任何人
需要域名/备案	否	**是**	否
HTTPS	可选	**必须**	隧道方提供
前后端形态	仍可用 dev server	**必须构建静态产物**	视隧道方式
承载真实数据	☑	⚠ 先修 §6.1	✕
长期可用	☑	☑	✕

---

3. 运行环境要求

3.1 硬件（引手册 §2.1）

项	最低	推荐	说明
CPU	2 核	4 核+	无 GPU 时走 CPU 推理 [实测] h=14 端到端 **23.5 s**（仍在 60 s 内）
内存	4 GB	8 GB+	[实测] 峰值常驻 **< 1 GB**（模型仅 31,729 参数）
磁盘	2 GB 可用	10 GB+	模型 128 KB；主要增长来自数据库与上传文件
GPU	可选	支持 CUDA 的任意显卡	[实测] 峰值显存仅 **111 MB**，2 GB 老卡也够

🔧 **结论：公开部署不一定要买 GPU 机器。** 一般云主机（4 核 8 GB）走 CPU 就能满足 60 s 预算。

代价是**并发能力更弱**（见 §7）—— GPU 版单任务 [实测] 11.85 s，CPU 版 23.5 s。

3.2 操作系统

- **Linux（Debian / Ubuntu 系）**：手册 §3 的生产部署路径即按此编写，**推荐**。
- **Windows**：本项目开发机就是 Windows，代码本身跨平台；但 systemd 换成「Windows 服务」或计划任务，

Nginx 用 Windows 版（或换 Caddy）。**若选 Windows，务必先读 §十二 的记忆条目里那几条 DOM/编码坑。**

3.3 依赖

- **Python 3.12**（`requirements.txt` 首行声明）
- ⚠ **torch 必须单独装**：`pip install torch==2.9.1+cu128 --index-url https://download.pytorch.org/whl/cu128`

—— 直接 `pip install -r requirements.txt` 会装成 CPU 版（判断依据：`/health` 的 `device` 字段）

- **Node.js 18+**：**只在构建前端时需要**，运行期不需要装
- 系统包：`python3.12-venv`、`sqlite3`、`nginx`、`curl`

3.4 网络与端口

端口	监听方	是否对外
443 / 80	Nginx	✅ 对外
8000	uvicorn	✕ **仅本机**（`127.0.0.1`）
5173	Vite dev server	✕ **生产环境不应存在**

3.5 磁盘与数据

- 数据库：单文件 SQLite（`data/app.db`），WAL 模式
- 上传：`data/uploads/<user_id>/`；导出：`data/exports/`
- 增长 [实测]：113 序列 × 14 天 ≈ 1,582 行明细 ≈ **180 KB / 任务**；单数据集上限 20 MB
- ⚠ 手册 §10.2 提示：`predictions` 表**超过 100 万行需重新评估**索引与分页

---

4. 部署步骤（生产形态）

4.1 Linux 服务器（照手册执行）

```
# ---- ① 取代码 ----
git clone https://github.com/LVLILOVE/ecommerce-sales-forecast.git /srv/sales-forecast
cd /srv/sales-forecast


# ---- ② Python 环境（手册 §2.3）----
python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
pip install torch==2.9.1+cu128 --index-url https://download.pytorch.org/whl/cu128


# ---- ③ 生产配置（🔒 不可照抄开发机的 .env）----
cp .env.example .env
#   必须改：APP_ENV=production / JWT_SECRET=新的随机长串 / CORS_ORIGINS=真实域名
python -c "import secrets;print(secrets.token_urlsafe(48))"   # 生成 JWT_SECRET


# ---- ④ 初始化数据库（自动建表 + 种子数据）----
python -c "from app.core.db import init_db; init_db()"


# ---- ⑤ 构建前端（🔒 dev server 不能对外，必须构建）----
cd web && npm ci && npm run build && cd ..
#   产物在 web/dist/ —— 绝对路径 /assets/…，必须挂在站点根路径
```

```
# ---- ⑥ 启动后端（交给 systemd，配置见手册 §6.2）----
sudo systemctl enable --now sales-forecast


# ---- ⑦ 配置 Nginx（手册 §3）+ HTTPS ----
sudo certbot --nginx -d forecast.example.com


# ---- ⑧ 自检 ----
curl -s https://forecast.example.com/health
```

⚠ **`.env`、`data/`、`outputs/models/` 都不在 git 里**（见 `.gitignore`）——

所以**模型权重和数据库需要用别的方式搬过去**（`scp` / `rsync`），否则启动自检会直接失败：

```
scp -r outputs/models user@server:/srv/sales-forecast/outputs/
scp data/app.db       user@server:/srv/sales-forecast/data/
```

> 启动自检是**刻意的强校验**（`BE-06`）：`ACTIVE_MODEL` 与数据库启用版本不一致、
>
> 或权重文件缺失，都会让**进程起不来** —— 这是为了防「服务起来了但加载的是错版本权重」这种静默故障。

4.2 Windows 服务器

代码路径一致，差异只在三处：

项	做法
服务托管	用 [NSSM](https://nssm.cc/) 把 `python -m uvicorn ...` 注册成 Windows 服务；或「任务计划程序 → 开机启动」
反向代理	Nginx for Windows，或用 **Caddy**（配置文件更短，自动 HTTPS）
路径	统一用绝对路径；⚠ 项目路径**不要含中文**（本机开发路径含中文，是能跑，但服务化后容易踩编码坑）

4.3 关键：前端必须构建，不能跑 dev server

	dev server（:5173）	生产构建（web/dist）
用途	开发热更新	对外服务
性能	无压缩无缓存	带 hash 长缓存 + gzip
安全	暴露源码与调试接口	只有产物
代理	内置 /api 代理	**由 Nginx 承担**

> 🟤 **别把 dev server 暴露到公网** —— 它会把源码、依赖预打包产物、HMR 接口全部暴露出去。

---

5. 权限控制

系统已经有一套完整的三层权限模型，**多用户访问不需要重新设计，只需要正确配置 + 补齐游客那一层。**

5.1 第一层：认证（你是谁）

机制	实现	说明
注册 / 登录	POST /api/v1/auth/*	bcrypt cost=12 存密码；JWT (HS256) 签发
Token 载体	Authorization: Bearer <token>	前端存 localStorage；401 时统一清空并跳登录
登录限流	**落库实现**（WS-2-05）	失败 3 次锁定 60 s（LOGIN_MAX_FAIL / LOGIN_LOCK_SECONDS）
未知邮箱限流	进程内内存镜像	防「用响应差异枚举已注册邮箱」
改密撤销	users.token_version	改密后全端登录态失效

机制	实现	说明
登出撤销	`jti` 黑名单（**进程内存**）	⚠ 重启后失效；多进程会失效（见 §7）

5.2 第二层：授权与数据隔离（你能碰谁的数据）

- **注册用户**：所有资源查询都过归属判定 → 别人的资源返回 **`40301`**（不是 404，避免资源探测）。

唯一真源：`app/services/result_service.py::_owned_task()`，历史/结果/对比/数据集四类接口共用。

- **管理员**：`users.role = 'admin'`，可跨用户操作，**是数据隔离红线的唯一显式例外**，

且**全操作写审计**（`operation_logs`）。

- **游客**：`user_id = 0` 共享桶 —— **这是目前唯一的隔离缺口，见 §6.1。**

5.3 第三层：功能门控

功能	是否要求登录	依据
上传 → 参数 → 执行 → 结果	✕ 不需要（游客可跑通）	`FR-USR-08`
下载导出（CSV / Excel）	☑ 需要	`API-14`-系列
历史记录 / 详情 / 对比	☑ 需要	`API-17`~`API-20`
管理端 `/api/v1/admin/*`	☑ 需要 `role = admin`	`CH-01`

5.4 🔵 公开暴露前必改清单

#	项	现状	必须改成
1	**游客数据隔离**	所有游客共享 `user_id = 0`	修 §6.1
2	**管理员默认密码**	`admin@salesforecast.cn` / `Admin@2026`（种子内置）	**立即改**；且系统**没有强制改密机制**，只能手动改
3	`JWT_SECRET`	开发机 `.env` 里的 48 字节	**重新生成**，且不要与开发机

#	项	现状	必须改成
		串	相同
4	APP_ENV	development	production
5	CORS_ORIGINS	只有 localhost	追加真实域名
6	HTTPS	无	**必须**（Token 走 Authorization 头，明文传输等于泄露登录态）
7	数据库内测试数据	可能有开发期测试账号 / 游客数据	清库或确认无敏感数据

---

6. 🔴 公开前必须处理的阻断项

6.1 【阻断级】游客之间没有任何数据隔离

证据链（代码级，可复核）：

1. 未登录用户被识别为 `GUEST_USER_ID = 0`

—— `app/core/deps.py::get_optional_user()` 在无 Token / Token 无效 / Token 过期时

一律返回 0，且不抛错（这是刻意的「降级」设计）。

2. 游客数据是「匿名共享」的，而且对已登录用户也开放

—— `app/services/dataset_service.py::assert_dataset_readable()`：

```
def assert_dataset_readable(dataset: Dataset, caller_id: int) -> None:
    """🔒 数据集读权限判定。**分支顺序不可交换。**"""
    if dataset.user_id == GUEST_USER_ID:
        return  # 游客数据匿名共享，任何调用者均可用（BE-41 第一分支）
    if dataset.user_id != caller_id:
        raise BizError(ErrorCode.FORBIDDEN, "无权访问该数据集")
```

第一分支**直接放行**：只要数据集归属 `user_id = 0`，**任何调用者（含已登录用户）都能读。**

3. 任务归属判定同样是「0 == 0」

—— `app/services/result_service.py:122`：

```
if int(task.user_id or 0) != int(user_id):
    raise BizError(ErrorCode.FORBIDDEN, "无权查看该任务的结果…")
```

两个游客的 `user_id` **都是 0 → 判定必然通过。**

4. 历史列表按 `user_id = 0` 过滤 —— `history_service.py::list_history()`：

游客打开历史页，**列出的就是全体游客的任务**，连枚举都不需要。

5. `tasks.id` 是 `INTEGER PRIMARY KEY AUTOINCREMENT`

—— 结果页天然可枚举（`/result/1`、`/result/2` …）。

为什么会这样（这一点很关键）：

这个设计的**本意**是「游客数据 = 系统内置的公共示例数据」—— 示例数据集（`API-07`）

不按用户复制，所有访客共用一份，所以共享是对的。

但代码没有区分「系统示例数据」与「访客自己上传的数据」 —— 两者都落在 `user_id = 0` 这一个桶里。

于是访客上传的**真实销售数据**（含仓库、商品、销量）自动变成了「公共数据」。

后果：公开部署后，访客 A 上传的数据，访客 B 打开历史页或遍历 ID 就能看到 ——

而且连注册用户也能看到。这是真实业务数据泄露，不是理论风险。

三种修法（按推荐度排序）：

方案	做法	改动范围	代价
① **关闭游客链路**（最省事，推荐给"真实用户"场景）	上传/参数/执行/结果四类接口的 `OptionalUserId` 全部改为 `CurrentUser`，**必须登录才能用**	接口层 4 处依赖声明	失去「免登录试跑」体验；**与 `FR-USR-08` 冲突，需回写 PRD 与技术文档**
② **匿名会话隔离**（保留免	首次访问下发不可猜的匿名	schema 加列（SQLite `ADD`	改动比 ① 大，但**保留产品**

方案	做法	改动范围	代价
登录体验）	标识（UUID4，HttpOnly Cookie）；`tasks`/`datasets` 新增 `guest_sid` 列；归属判定改为「`user_id` 相等 **且**（非游客 或 `guest_sid` 相等）」	`COLUMN` 可行）+ `_owned_task` / `assert_dataset_readable` 两处真源 + 上传与列表的写入处	**设计**
③ 明确声明「请勿上传真实数据」并定期清库	不改代码，靠公告与运营约束	0	**不解决，只规避** —— 公开场景下不可接受

🔪 **为什么 ① 是"干净"的**：改成必须登录后，`user_id = 0` 这个桶里就**只剩下系统内置的示例数据**（`API-07` 那份本来就应该对所有人可见）—— 于是「匿名共享」回归它原本的语义，不需要再区分「示例数据」与「访客上传数据」。

选 ② 的话要多做一步：把 `guest_sid` 为空的记录视为公共示例数据，非空的按会话隔离。

📌 改动成本提示：归属判定有**唯一真源**（`result_service._owned_task` + `dataset_service.assert_dataset_readable`），所以两种方案都**不需要**逐个接口改 —— 这是前期"单一真源"设计的回报。

📌 **如果只是想给面试官演示**：用 ① —— 「必须注册才能使用」本身也是加分项（体现真实产品意识）。

如果这是课程交付、要求「多用户可用」：建议 ②，并在答辩时把它作为「发现并修复的隔离缺陷」讲出来。

6.2 【高】匿名用户可以无限消耗算力

现状：全项目**除登录限流外没有任何业务配额**（已核查 `app/` 全量代码）。

游客无需登录即可上传并触发预测 —— 单次任务占用 12~24 s 的推理时间（`[实测]`）。

后果：公开后任何人可脚本化批量提交，把服务排队占满 —— **这是可用性问题，也是成本问题。**

修法（按投入排序）：

措施	说明
登录才能跑任务	与 §6.1 方案 ① 合并解决，一举两得
按 IP / 会话限速	Nginx `limit_req` 层即可实现，**不改代码**： `limit_req_zone $binary_remote_addr zone=api:10m rate=10r/m;`
每用户并发/日配额	需新增代码与表结构，改动最大，但最彻底
注册开关	若只想给特定人用，干脆**关闭开放注册**、由管理员开号

6.3 【高】默认管理员账号

`code/db/seed.sql` 内置 `admin@salesforecast.cn` / `Admin@2026`，

且代码中**没有「首次登录强制改密」机制**（已核查，无相关实现）。

上线前必须手动修改，否则等于把管理端（可见全站用户、可重置任意用户密码、可硬删除任意任务）的公钥挂在网上。

6.4 【中】配置项

`JWT_SECRET`（重新生成）、`APP_ENV=production`、`CORS_ORIGINS`（真实域名）、HTTPS —— 见 §5.4 清单。

6.5 【中】单进程带来的并发上限

见 §7。**这与「多人访问」直接相关**：不是「配好就能上量」，而是有明确天花板。

7. 容量与并发（单 worker 的现实）

7.1 为什么必须是单进程

手册 §6.2 与 `app/core/security.py` 都写明：**`--workers` 固定为 1。**

原因：**登录限流的未知邮箱计数**与**登出 `jti` 黑名单**都是**进程内内存**实现 ——

多 worker 会各持一份，限流与登出撤销**形同虚设。**

> 🔒 **这不是可以省略的前提**：要上多进程/多实例，**必须先**把这两项迁到 Redis 或数据库。

7.2 实测并发能力（手册 §10.1）

场景	[实测]	判读
单任务（GPU，h=14，113 序列）	11.85 s	顺畅
单任务（**无 GPU 走 CPU**）	23.49 s	仍满足 60 s
3 任务并发	32.2 / 37.6 / 37.5 s	**全部成功、锁冲突 0**，但每个都慢了约 3 倍
极限（1000 序列 × h=30）	250.3 s → `E_TIMEOUT`	**软失败**：无 5xx、无半成品

判读：

- 推理本身只要 **21 ms** —— 慢在**预处理 / 特征 / 指标**，是 CPU 密集且受 GIL 限制；
- **单进程内任务实际是串行的**：3 个人同时点「开始预测」，第 3 个人要等到约 40 s；
- **10 人以上同时用会明显排队。**这不是配置问题，是架构约束。

7.3 扩容路径（按优先级）

步骤	动作	前置条件
1	**先加 CPU 核数 / 换更强单核**	无
2	**任务队列**：提交即返回 `task_id`，后台串行消费，前端轮询状态	⚠ **当前是同步语义**（响应返回时任务已入终态，不存在运行期轮询窗口）。改异步是一次真实的产品行为变更，需回写 PRD
3	**多实例 + 共享存储**	**必须先把限流与 `jti` 黑名单迁到 Redis**，且数据库迁移到 PostgreSQL（SQLite 单写者，多进程写会锁冲突）

📌 **诚实结论**：以当前架构，**几十人规模的内部使用没问题，面向公网开放注册并期待高并发不现实。**

对外发布时建议配合「关闭开放注册 / 白名单邀请」来控制规模。

---

8. 运维要点（交叉引用手册）

事项	手册位置	多人访问下的额外注意
数据备份	§7	⚠ **不能直接 `cp` WAL 模式的 `.db`** —— 必须用 SQLite 在线备份 API，备份后跑完整性校验
临时文件清理	§8	导出产物 TTL 7 天；**游客数据 24 h**（`GUEST_DATA_TTL_HOURS`）—— 公开后清理压力更大
监控告警	§10.3	新增两条：**`42901` 频繁触发**（可能在撞库/刷接口）、**磁盘水位 80%**
日志	§11	已内置 `request_id` 串联 + 敏感字段遮蔽；对外发布建议接入集中日志

事项	手册位置	多人访问下的额外注意
审计	`operation_logs`	管理端操作全留痕；**公开前确认审计表不会被游客数据淹没**
升级回滚	§12	回滚**必须**先备份（手册明确标注不可省）

---

9. 上线检查清单

环境

- ☐ 服务器满足 §3.1（CPU 版即可，无需 GPU）
- ☐ Python 3.12 + `requirements.txt` + **单独装的 CUDA/CPU 版 torch**
- ☐ Node 18+（**仅构建前端时**需要）
- ☐ 防火墙只放行 80 / 443；**8000 不对外**
- ☐ `outputs/models/` 与 `data/app.db` 已搬到服务器（**不在 git 里！**）

配置

- ☐ `APP_ENV=production`
- ☐ `JWT_SECRET` = **新生成的** ≥32 字符串
- ☐ `CORS_ORIGINS` 含真实域名
- ☐ HTTPS 已启用且证书自动续期
- ☐ **管理员密码已改掉** `Admin@2026`

发布

- ☐ 前端已 `npm run build`，由 Nginx 托管 `web/dist`（**不是 dev server**）
- ☐ Nginx：`client_max_body_size 25m`、`proxy_read_timeout 90s`、`index.html` 不缓存

☐ 后端 systemd 托管，`--host 127.0.0.1 --workers 1`

☐ `/health` 返回 `model_loaded: true`

安全（⚫ 缺一不可）

☐ **§6.1 游客隔离已处理**（关闭游客链路 **或** 匿名会话隔离）

☐ **§6.2 匿名滥用已加闸**（登录门槛 / Nginx 限速 / 配额）

☐ 已确认「非管理员用户无法访问他人资源」（用两个账号互相访问验证返回 `40301`）

☐ 备份策略已就位并**演练过一次恢复**

演示/交付

☐ 用**两个不同账号**验证过隔离（A 看不到 B 的任务）

☐ 用**未登录浏览器**验证过 §6.1 的处理结果

☐ 确认无开发期测试数据残留在生产库

附录：与上游文档的关系

文档	关系
《部署运维手册 V1.0》	**主文档。**本文只补充「对外暴露 + 多用户」维度，安装/启动/备份/升级一律以手册为准
《PRD V1.0》	§6.1 若关闭游客链路，需回写 `FR-USR-08`；§7.3 若改任务队列，需回写同步语义相关条款
《开发技术文档 V1.0》	归属判定唯一真源 `_owned_task`；`--workers 1` 的推理由 `security.py` docstring 给出
《数据库设计文档 V1.0》	§6.1 方案 ② 需新增 `tasks.guest_sid` / `datasets.guest_sid` 列
《测试报告 V1.0》	§10.1 的并发与容量基线取自该报告的实测记录

*编制：2026-10-15 · 基线：工作区当前代码 + 手册 V1.0 · 风险等级：§6.1 为**阻断级**，未处理不得对外发布*